

Project 3a: Virtual Memory

Preliminaries

Fill in your name and email address.

钟骏宇 2400012946@stu.pku.edu.cn

If you have any preliminary comments on your submission, notes for the TAs, please give them here.

Please cite any offline or online sources you consulted while preparing your submission, other than the Pintos documentation, course text, lecture notes, and course staff.

Page Table Management

DATA STRUCTURES

A1: Copy here the declaration of each new or changed struct or struct member, global or static variable, typedef, or enumeration. Identify the purpose of each in 25 words or less.

在 `vm/page.h` 中:

```
enum vm_page_type {
    VM_PAGE_FILE,
    VM_PAGE_ZERO,
    VM_PAGE_STACK,
    VM_PAGE_SWAP
};

struct vm_page {
    void *upage; /* 用户虚拟页地址。 */
    enum vm_page_type type; /* 该页的后备存储类型。 */
    bool writable; /* 用户代码是否可写该页。 */
    bool loaded; /* 该页当前是否驻留在内存中。 */
    struct hash_elem elem; /* 每进程 SPT 的哈希表节点。 */

    struct file *file; /* 可执行文件后备页对应的文件。 */
    off_t ofs; /* 在后备文件中的偏移。 */
    uint32_t read_bytes; /* 首次装入时需要读取的字节数。 */
    uint32_t zero_bytes; /* 文件内容之后需要清零的字节数。 */

    size_t swap_index; /* 若被换出，对应的 swap slot 编号。 */
    bool busy; /* 页面正在装入或换出时置为 true。 */
};
```

- `enum vm_page_type`: 区分文件后备页、swap 页、零页和栈页。
- `struct vm_page`: 保存单个用户虚拟页的补充 meta data 与后备来源。

在 `vm/frame.c` 中:

```

struct frame_entry {
    void *kpage;           /* 该 frame 的内核虚拟地址。 */
    void *upage;           /* 当前映射到的用户虚页地址。 */
    struct thread *owner;  /* 拥有该映射页面的进程。 */
    bool pinned;           /* 敏感 I/O 期间禁止被换出。 */
    bool busy;             /* 正在换出或复用时置为 true。 */
    struct list_elem elem; /* 全局 frame table 的链表节点。 */
};

```

- `struct frame_entry`: 跟踪一个用户池 frame 及其当前拥有者。

在 `vm/swap.c` 中:

```

static struct block *swap_block;
static struct bitmap *swap_map;
static struct lock swap_lock;

```

- `swap_block`: 指向当前承担 swap 角色的块设备。
- `swap_map`: 记录哪些 swap slot 已经被占用。
- `swap_lock`: 串行化 swap slot 的分配与回收。

在 `threads/thread.h` 中:

```

struct hash spt;           /* 每个进程自己的补充页表。 */
void *saved_esp;           /* 最近一次记录的用户栈指针。 */

```

- `spt`: 保存该进程拥有的全部用户虚拟页 meta data。
- `saved_esp`: 用于判断一次 fault 是否属于合法的栈增长。

在 `vm/frame.c` 中:

```

static struct list frame_table;
static struct list_elem *clock_hand;
static struct lock frame_lock;

```

- `frame_table`: 保存所有用户 frame 的全局链表。
- `clock_hand`: clock 换出算法当前扫描到的位置。
- `frame_lock`: 串行化 frame 分配与换出过程中的 meta data 修改。

在 `threads/thread.c` 中:

```

static bool vm_ready;

```

- `vm_ready`: 确保初始线程的 SPT 在内存系统可用后再初始化。

ALGORITHMS

A2: In a few paragraphs, describe your code for accessing the data stored in the SPT about a given page.

每个进程都拥有一个自己的补充页表，采用哈希表实现，并以向下取整后的用户虚拟页地址作为键。在装载程序时，`load_segment()` 不再像 Lab 2 那样立即分配 frame 并把数据读入内存，而是为该 segment 中的每个虚拟页创建一个 `struct vm_page`，然后插入当前线程的 SPT。这个条目记录了将来真正发生 page fault 时需要用到的全部信息，例如后备文件、文件偏移、需要读取的字节数、需要补零的字节数，以及页面是否可写。

当发生 page fault 时，`page_fault()` 会先检查 fault address 是否属于用户地址空间，再在当前线程的 SPT 中查找对应条目。若查找失败，但 fault 地址满足栈增长启发式条件，则先在 SPT 中登记一个新的栈页。随后，内核根据页面类型决定如何恢复该页：若页面是文件后备页，则按记录的偏移从文件中读入指定字节并把剩余部分清零；若页面此前已被换出到 swap，则从 swap 设备中读回；若页面是零页或栈页，则在分配页框时直接得到一页清零内存。页面装入后，内核通过 `pagedir_set_page()` 安装映射，并把 `loaded` 置为 true。

同一张 SPT 还会在进程退出时被用来做资源回收。内核不会只依赖硬件页表，而是遍历 SPT 来找到该进程拥有的每一个用户页。对于每一页，销毁逻辑会判断它是否仍然对应某个 swap slot，并在必要时释放该 slot，最后释放页 meta data。

A3: How does your code coordinate accessed and dirty bits between kernel and user virtual addresses that alias a single frame, or alternatively how do you avoid the issue?

通过尽可能避免使用 kernel alias 来规避 accessed bit 和 dirty bit 的别名问题。一个用户页一旦建立映射，内核就始终把用户虚拟地址视为访问位与脏位检查的唯一权威地址。尤其在页面淘汰时，替换算法检查的是拥有者页目录中的用户映射，而不是对应的内核别名地址。

由于所有替换决策都通过拥有者进程的页目录读取 accessed bit 和 dirty bit，clock 算法看到的正是 CPU 在用户态真实更新的那份状态，无需手工合并多个 alias 上的位信息。

SYNCHRONIZATION

A4: When two user processes both need a new frame at the same time, how are races avoided?

frame 分配由一个全局的 `frame_lock` 串行化。某个进程需要新 frame 时，会先获取该锁，然后要么直接从用户池中分配空闲页，要么执行页面淘汰流程来腾出一个 frame，然后释放锁再让下一个进程去获得新 frame。

SPT 本身是每进程私有的数据结构，因此不同进程之间围绕页 meta data 的 race 有限。真正全局共享的主要是 frame table 与 swap slot 位图，而它们分别由 `frame_lock` 和 `swap_lock` 保护。所以当两个 page fault 几乎同时发生时，最关键的竞争会在 frame 分配这一边界上被正确串行化处理。

RATIONALE

A5: Why did you choose the data structure(s) that you did for representing virtual-to-physical mappings?

添加了每个进程的 hash table（SPT）辅助管理虚拟页到物理页的映射，主要原因是最常见、最关键的操作就是在 page fault 发生时根据虚拟页地址做点查询。哈希表在这一场景下查找效率较高，内存占用少。同时，它在进程退出时也便于整体遍历并释放所有条目。

Paging To And From Disk

DATA STRUCTURES

B1: Copy here the declaration of each new or changed struct or struct member, global or static variable, typedef, or enumeration. Identify the purpose of each in 25 words or less.

```
struct vm_page {
    enum vm_page_type type; /* FILE、SWAP、ZERO 或 STACK。 */
    bool loaded; /* 当前是否驻留在某个 frame 中。 */
    struct file *file; /* 后备可执行文件。 */
    off_t ofs; /* 惰性装载时使用的文件偏移。 */
    uint32_t read_bytes; /* 从文件读取的字节数。 */
    uint32_t zero_bytes; /* 文件数据后需要清零的字节数。 */
    size_t swap_index; /* 被换出后所在的 swap slot。 */
    bool busy; /* 防止并发装页和换页冲突。 */
};

struct frame_entry {
    void *kpage; /* 实际的用户池物理 frame。 */
    void *upage; /* 当前映射到该 frame 的用户虚页。 */
    struct thread *owner; /* 拥有该映射页面的进程。 */
    bool pinned; /* 被 pin 后不可参与淘汰。 */
    bool busy; /* 正在换出或装填。 */
    struct list_elem elem; /* 全局 frame table 的链表节点。 */
};

static struct block *swap_block;
static struct bitmap *swap_map;
static struct lock swap_lock;
```

ALGORITHMS

B2: When a frame is required but none is free, some frame must be evicted. Describe your code for choosing a frame to evict.

代码里采用全局 clock 算法，它是对 LRU 的一种经典近似。frame table 以全局链表形式维护，而 `clock_hand` 记录下一轮扫描应从哪里开始。当用户池中已经没有空闲 frame 时，内核就从 `clock_hand` 指向的位置开始扫描整个 frame table。

pinned 或 busy 的 frame 会被立即跳过，因为淘汰它们会破坏正在进行的 I/O 或 page fault 处理。对于其他 frame，内核检查其拥有者页目录中对应用户映射的 accessed bit。若该位为 1，则将其清零并给予第二次机会；若该位已经为 0，则把该 frame 选为 victim。

B3: When a process P obtains a frame that was previously used by a process Q, how do you adjust the page table (and any other data structures) to reflect the frame Q no longer has?

在选出 victim frame 之后，内核首先把对应的 victim frame 标记为 `busy` 和 `pinned`，并把 victim page 标记为 `busy`，避免其他线程与当前换出流程竞争。随后，内核使用 `pagedir_clear_page()` 将 Q 的页表中这条用户映射清除。此时，Q 已经不再拥有一个指向该 frame 的 present 映射。

接着，内核判断 Q 的页面在被淘汰后应当迁移到哪里。如果它是一个干净的、由可执行文件支持的页面，那么只需把它标记为 non-resident 并保持 file-backed 即可，因为将来仍可直接从可执行文件重新读取；如果该页已经变脏，或者其内容无法安全地仅靠文件恢复，则会该页写入新分配的 swap slot，并把页面 meta data 更新为 `VM_PAGE_SWAP` 类型，同时记录相应的 `swap_index`。

最后，旧页面对应的 `vm_page` 会被标记为 not loaded，而 victim 对应的 `frame_entry` 会被原地复用：它的 `owner` 被更新为 P，`upage` 被更新为 P 需要的新虚页地址，并在必要时按 `PAL_ZERO` 标志清零页框内容。随后，该 frame 再被新页面装填并重新安装到 P 的页表中。

SYNCHRONIZATION

B5: Explain the basics of your VM synchronization design. In particular, explain how it prevents deadlock. (Refer to the textbook for an explanation of the necessary conditions for deadlock.)

我们的 VM 设计采用少量锁，而不是一把覆盖整个虚拟内存子系统的大锁。frame table 由 `frame_lock` 保护，swap slot 分配位图由 `swap_lock` 保护，文件系统则继续使用原有的全局文件系统锁。页面级别的状态位，例如 `busy` 和 `pinned`，用于在不引入新的全局瓶颈的前提下阻止并发干扰。

从死锁的四个必要条件来看：

在涉及 `filesys_lock` 的路径上，系统调用在获取锁之前先通过 `pin_buf()/pin_str()` 确保所有用户页均已驻留，持有 `filesys_lock` 期间无需再等待任何其他锁，所以不满足 hold and wait 必要条件；

在 `frame_lock` → `swap_lock` 路径上，我们维护严格的加锁顺序：`frame_lock` 先于 `swap_lock`，且两者均不与 `filesys_lock` 同时持有，不存在形成等待环路的可能。所以不满足 circle wait 必要条件。

B6: A page fault in process P can cause another process Q's frame to be evicted. How do you ensure that Q cannot access or modify the page during the eviction process? How do you avoid a race between P evicting Q's frame and Q faulting the page back in?

在换出开始之前，victim frame 及其对应页面都会在 `frame_lock` 保护下被标记为 `busy`。随后，在 frame 被复用之前，Q 的页表中这条映射会被清除。一旦映射被移除，Q 之后再访问这个虚拟地址时就只会重新触发 page fault，而不可能继续直接访问旧的 frame 内容。

`busy` 标志用于阻止换出与重新 fault in 之间的竞争。若 Q 在换出仍在进行时再次 fault 到这个页面，`page_load()` 会发现 `busy` 为真，`thread_yield()` 让出 CPU 等待换出完成，而不会装入第二份副本。只有换出线程完成 meta data 更新、页面状态稳定后，Q 才能重新 fault in。

B7: Suppose a page fault in process P causes a page to be read from the file system or swap. How do you ensure that a second process Q cannot interfere by e.g. attempting to evict the frame while it is still being read in?

当某个 fault handler 为页面分配好 frame 后，在开始 I/O 之前会先把页面本身标记为 `busy`。对于已经在使用中的 frame，替换算法本来就会跳过 `busy` 页面对应的 frame，因此即使当前内存压力很大，也不会去复用内容仍在读取中的页框。

只有当 I/O 完成、页表项安装成功、页面 meta data 全部更新完毕之后，内核才会清除 `busy` 标志，保证没有线程会去淘汰一个内容仍在加载中的 frame。

B8: Explain how you handle access to paged-out pages that occur during system calls. Do you use page faults to bring in pages (as in user programs), or do you have a mechanism for "locking" frames into physical memory, or do you use some other design? How do you gracefully handle attempted accesses to invalid virtual addresses?

我们采用主动 pin 机制。系统调用在进入文件系统 I/O 临界区之前分两步处理用户缓冲区：第一步，`check_ptr()` / `check_buf()` / `check_str()` 通过 `page_resolve()` 验证地址合法，若某页尚未驻留但有合法 SPT 条目，这一步就已触发补页；第二步，`pin_buf()` / `pin_str()` 调用 `page_resolve_and_pin()` 将相关 frame 锁住，确保在持有文件系统锁期间这些页不会被淘汰、不会触发嵌套 fault。I/O 完成后统一 unpin。

对于非法虚拟地址，内核在第一步的 check 阶段就会提前拒绝（`page_resolve()` 返回 false），调用 `syscall_exit(-1)` 终止进程；若非法访问发生在用户代码中，page fault handler 判断该地址既无 SPT 条目也不符合栈增长规则，同样以 -1 终止进程，并经由正常清理路径回收所有资源。

RATIONALE

B9: A single lock for the whole VM system would make synchronization easy, but limit parallelism. On the other hand, using many locks complicates synchronization and raises the possibility for deadlock but allows for high parallelism. Explain where your design falls along this continuum and why you chose to design it this way.

我们的设计更粗粒度，但不是单一大锁，而是只为真正全局共享的资源各设一把锁（`frame_lock`、`swap_lock`、`filesys_lock`），辅以 `pinned/busy` 状态位处理页面级的并发冲突。

这样选择的原因在于虚拟内存中大多数状态根本不需要全局锁。SPT 是每进程私有的哈希表，不同进程之间没有共享，无需加锁；页表也是每进程独立的。真正需要全局串行化的只有 frame table（物理页分配）和 swap slot 位图这两处，三把粗粒度锁恰好覆盖了所有跨进程共享点，既避免了单一大锁把无关操作强行串行化，又不必引入逐页锁带来的复杂度和死锁风险。